TrailBlazer | Advanced Topics

Reflection on research and implementation

1. Game and Level Design

Why we chose this topic

We chose level design because TrailBlazer depends on the quality of its puzzles. The player leaves directional arrows while moving, then uses them to guide robots onto pressure plates and unlock an exit. Implementing these rules creates a working game, but arranging them into understandable and interesting challenges requires a separate design process.

What we learned from the research

Tim Ryan's *Beginning Level Design, Part 1* helped us think about levels from the player's perspective. His discussion connects challenge to the game's central skills, stresses the value of simple rules and variation, and warns against frustrating or unsolvable situations [1]. For TrailBlazer, we interpret this as testing route planning and the order of robot activation. More obstacles alone do not necessarily create a better puzzle; their placement should make the player reason about the mechanics.

How this appears in our project

The project contains four separate maps. The first has one robot and one matching plate, while the fourth contains four robots and four plates. This creates a concrete progression from understanding one route to coordinating several routes and activation steps. Obstacles restrict available paths, and the same arrow mechanic can produce different puzzles through changes in placement. The introduction explains the rules, while restart and solution controls support retrying a puzzle and inspecting a possible route.

Challenges, decisions and what we learned

The main design challenge is that a map can be valid without being a good puzzle. The code checks structural conditions, including a player, an exit and matching robot and plate colours, but these checks do not prove that a solution exists or that the difficulty is appropriate. The saved solution layouts provide useful reference routes; they do not establish how a new player will experience a level. Our main lesson is to separate technical correctness from player understanding. We would improve the next iteration by asking new players to explain their plans and recording where they become confused. The progression is visible in the implementation, but we cannot conclude that it is well balanced without that feedback.

[1] Tim Ryan. *Beginning Level Design, Part 1*. Game Developer, 16 April 1999. gamedeveloper.com/design/beginning-level-design-part-1

2. Git Branching Workflows

Why we chose this topic

We chose Git branching workflows because our game and its documentation change alongside one another. We need a way to combine contributions while retaining a history of what changed. Understanding a workflow is relevant beyond knowing how to commit files: it determines when changes are shared, how work is integrated and how the team recovers from mistakes.

What we learned from the research

Atlassian's comparison explains that a workflow should suit the team and that published approaches can be adapted rather than followed as fixed rules [2]. We took from this that our process should remain simple enough to use consistently. Branches allow work to proceed separately, while merging brings it into a shared version. A useful workflow also needs agreement about when to synchronise and how to integrate changes; merely creating branches does not provide that agreement.

How this appears in our project

Our repository uses a shared main branch and a development branch named EdiBS86. The history records pull-request merges from that branch, including pull requests #7, #8 and #9, alongside changes to robot movement, menus, selectable levels and the finished game. This shows a branch-and-merge approach in practice. There are also direct commits to main, including backlog updates, so we would describe the actual process as a mixture rather than a strict implementation of one named workflow. The history confirms integration through pull requests, but it does not by itself prove that each change received a formal review.

Challenges, decisions and what we learned

A recent backlog update made the importance of synchronising clear. Its push was rejected because GitHub already contained newer commits. The update also encountered an open Word document that prevented Git from replacing the backlog file. The eventual integration preserved the backlog commit and merged the newer game changes; a safety backup retained the interrupted working-file changes. This experience helped us distinguish a saved local commit from a successfully published change. It also showed that document locks and local file state can affect version-control work even when the game code is correct.

Impact and improvements for future work

The workflow gives our project a shared version and a traceable record of development. For future work, we would use short-lived branches named after specific tasks, make focused commits and synchronise before starting and before merging. We would also agree on a small pre-merge check, such as launching the game and checking the affected level. These are improvements we would adopt, rather than checks we can claim were consistently followed already. Our main lesson is that branching supports collaboration only when the team also follows a clear and repeatable integration process.

[2] Atlassian. *Comparing Git workflows: What you should know*. Atlassian Git Tutorials. atlassian.com/en/git/tutorials/comparing-workflows